

Consulta de Endereço por CEP

Ficha de Atividade Prática | Individual

Contexto

Uma equipe recebeu a tarefa de criar uma aplicação web para consultar endereços brasileiros. Na primeira página, o usuário informa CEP e cidade. Na segunda, o sistema consulta o ViaCEP, confere a cidade e apresenta o endereço disponível e um link para o Google Maps.

O CEP não identifica o número de uma residência. O resultado e o mapa correspondem ao endereço associado ao CEP, sem garantir a localização de uma casa específica. Alguns CEPs não possuem logradouro ou bairro cadastrados.

Objetivo

Planejar e desenvolver uma aplicação com HTML, CSS e JavaScript, aplicando levantamento de requisitos, modelagem UML, planejamento de UX, consumo de API, testes e versionamento com Git.

Etapas da atividade

1. Organize os requisitos no Trello antes de programar.
2. Elabore o diagrama de casos de uso e os esboços das duas páginas.
3. Desenvolva as páginas e integre os códigos JavaScript apresentados.
4. Teste validações, consulte, mensagens e abertura do Google Maps.
5. Registre as evidências e envie a versão final para a branch main.

Comando: implemente o fluxo completo e esteja preparado para explicar individualmente as decisões do projeto e o funcionamento do código.

1 Levantamento de requisitos

Crie no Trello as listas Requisitos Funcionais, Requisitos Não Funcionais, Regras de Negócio e Dúvidas ou Pendências. Cada cartão deve conter identificador, descrição clara e condição de aceite. Use “O sistema deve...” nos requisitos.

Tipo	Especificação mínima
RF01	Receber CEP e cidade e encaminhar os dados para resultado.html.
RF02	Consultar o ViaCEP pelo CEP e apresentar CEP, logradouro, complemento, bairro, cidade, estado e UF.
RF03	Informar erros e divergência de cidade; permitir uma nova consulta.
RF04	Disponibilizar o link do Google Maps após uma consulta válida.
RNF01	Permitir uso em celular e computador, sem rolagem horizontal a 360 px.
RNF02	Usar rótulos nos campos, navegação por teclado e mensagens legíveis.
RN01	Aceitar CEP com oito números, com ou sem hífen; cidade obrigatória.
RN02	Comparar cidade ignorando acentos, caixa e espaços duplicados. Em divergência, bloquear resultado e link.
RN03	Exibir “Não informado” para campos ausentes; não inventar número de imóvel.

Exemplo de aceite do RF01: ao enviar 01001-000 e São Paulo, a página de resultado recebe cep=01001000 e a cidade informada na URL.

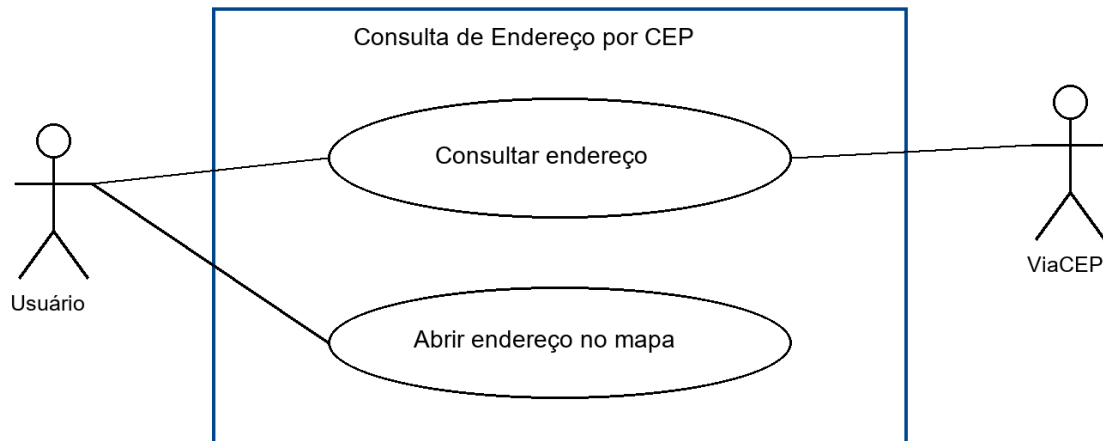
Planejamento de UX

Desenhe dois esboços antes de implementar. Na página de entrada, posicione título, rótulos CEP e Cidade, exemplos de preenchimento, botão Consultar Endereço e área de mensagem. Na página de resultado, organize os campos retornados, o endereço reunido, o link Abrir no Google Maps e Fazer nova consulta.

Represente os estados de carregamento, sucesso, CEP inválido, CEP inexistente, cidade divergente e falha de conexão. Garanta foco visível, contraste adequado e mensagens que expliquem como corrigir o problema. Salve os esboços em docs/word.

2 Diagrama de casos de uso

Elabore um diagrama com o ator Usuário e o serviço externo ViaCEP fora da fronteira do sistema. Use elipses para os objetivos e associações com linhas simples. O exemplo abaixo orienta a estrutura; entregue seu diagrama e descreva o fluxo principal e as alternativas.



Consultar endereço inclui a validação dos dados, a consulta ao ViaCEP, a conferência da cidade e a apresentação do resultado. Digitar um campo e chamar fetch são passos do fluxo, não objetivos independentes do usuário.

Abrir endereço no mapa só fica disponível após sucesso. A aplicação abre uma URL externa do Google Maps. Não é necessário acrescentar include ou extend apenas para indicar a sequência entre telas.

Estrutura do projeto

```

Estrutura do projeto
consulta-endereco/
  index.html      # entrada de CEP e cidade
  resultado.html  # consulta e apresentação do resultado
  style.css       # estilos compartilhados
  index.js        # validação e navegação
  resultado.js    # ViaCEP, cidade e Google Maps
  README.md       # execução, links e testes
  docs/          # requisitos, diagrama e esboços
  
```

Crie os documentos HTML com lang="pt-BR", charset="UTF-8" e meta viewport. Vincule style.css nas duas páginas. Os trechos a seguir entram no body; desenvolva o restante do HTML e o CSS.

3 Primeira página

Em index.html, utilize estes identificadores para conectar o formulário ao JavaScript. Salve todos os arquivos em UTF-8.

```
index.html

<form id="formConsulta">
  <label for="cep">CEP</label>
  <input id="cep" type="text" inputmode="numeric"
    maxlength="9" placeholder="01001-000" required>
  <label for="cidade">Cidade</label>
  <input id="cidade" type="text"
    placeholder="São Paulo" required>
  <button type="submit">Consultar Endereço</button>
</form>
<p id="mensagem" role="status" aria-live="polite"></p>
<script src="index.js"></script>
```

Código de index.js

```
Código de index.js

const form = document.getElementById("formConsulta");
const mensagem = document.getElementById("mensagem");

form.addEventListener("submit", (event) => {
  event.preventDefault();
  const entrada = document.getElementById("cep").value.trim();
  const cidade = document.getElementById("cidade")
    .value.trim().replace(/\s+/g, " ");
  mensagem.textContent = "";

  if (!/^\d{5}-?\d{3}$/.test(entrada)) {
    mensagem.textContent = "Informe um CEP com 8 números.";
    return;
  }
  if (!cidade) {
    mensagem.textContent = "Informe a cidade.";
    return;
  }
  const cep = entrada.replace("-", "");
  const parametros = new URLSearchParams({ cep, cidade });
  window.location.href = `resultado.html?${parametros}`;
});
```

Entenda o código

preventDefault evita o envio padrão do formulário. A expressão regular aceita oito números ou o formato 00000-000; letras e outros formatos são rejeitados. URLSearchParams codifica CEP e cidade para transporte na URL.

4 Segunda página e validação

Em resultado.html, insira o trecho abaixo. O link permanece oculto até o sucesso da consulta.

```
resultado.html

<h1>Resultado da consulta</h1>
<div id="resultado" role="status" aria-live="polite"></div>
<a id="linkMaps" hidden target="_blank"
  rel="noopener noreferrer">Abrir no Google Maps</a>
<p><a href="index.html">Fazer nova consulta</a></p>
<script src="resultado.js"></script>
```

Código de resultado.js parte 1 de 3

Copie as três partes, na ordem, no mesmo arquivo resultado.js.

```
Código de resultado.js parte 1 de 3

const parametros = new URLSearchParams(window.location.search);
const cep = parametros.get("cep") || "";
const cidadeInformada = parametros.get("cidade") || "";
const resultado = document.getElementById("resultado");
const linkMaps = document.getElementById("linkMaps");

function normalizar(texto) {
  return texto.normalize("NFD")
    .replace(/[\u0300-\u036f]/g, "")
    .toLowerCase().trim().replace(/\s+/g, " ");
}

function mostrarCampo(rotulo, valor) {
  const linha = document.createElement("p");
  const titulo = document.createElement("strong");
  titulo.textContent = `${rotulo}: `;
  linha.append(titulo, valor || "Não informado");
  resultado.append(linha);
}

async function consultarEndereco() {
  linkMaps.hidden = true;
  if (!/^\d{8}$/.test(cep) || !cidadeInformada.trim()) {
    resultado.textContent = "Dados inválidos. Volte e informe CEP e cidade.";
    return;
  }
  resultado.textContent = "Consultando endereço...";
  const controle = new AbortController();
  const limite = setTimeout(() => controle.abort(), 10000);
```

Entenda o código

A segunda página valida também os parâmetros da URL, inclusive no acesso direto. normalizar permite comparar “São Paulo” com “sao paulo”. mostrarCampo insere texto sem interpretar conteúdo recebido como HTML.

5 Consulta ao ViaCEP e conferência da cidade

Código de resultado.js parte 2 de 3

Código de resultado.js parte 2 de 3

```
try {
  const resposta = await fetch(
    `https://viacep.com.br/ws/${cep}/json/`,
    { signal: controle.signal }
  );
  if (!resposta.ok) {
    throw new Error("Falha HTTP");
  }
  const dados = await resposta.json();
  if (dados.erro) {
    resultado.textContent = "CEP não encontrado.";
    return;
  }
  if (!dados.localidade || !dados.uf || !dados.cep) {
    throw new Error("Resposta incompleta");
  }
  if (normalizar(cidadeInformada) !==
    normalizar(dados.localidade)) {
    resultado.textContent =
      `Este CEP pertence a ${dados.localidade}/${dados.uf}, ` +
      `e não a ${cidadeInformada}. Faça uma nova consulta.`;
    return;
  }

  resultado.replaceChildren();
  const campos = [
    ["CEP", dados.cep],
    ["Logradouro", dados.logradouro],
    ["Complemento", dados.complemento],
    ["Bairro", dados.bairro],
    ["Cidade", dados.localidade],
    ["Estado", dados.estado],
    ["UF", dados.uf]
  ];
  campos.forEach(([rotulo, valor]) => {
    mostrarCampo(rotulo, valor);
  });
}
```

Como a integração funciona

fetch envia uma requisição GET para o endereço do serviço. await aguarda a resposta; resposta.ok verifica o status HTTP e resposta.json() interpreta o JSON. O indicador erro informa que o CEP não foi encontrado. A consulta usa apenas o CEP; a cidade digitada serve para conferir o município retornado.

O código diferencia CEP inexistente de falha técnica. Se a cidade divergir, informa o município correto e encerra o fluxo sem liberar o link do mapa. Campos sem conteúdo recebem “Não informado”.

6 Endereço completo e Google Maps

Código de resultado.js parte 3 de 3

Código de resultado.js parte 3 de 3

```
const endereco = [
  dados.logradouro, dados.complemento, dados.bairro,
  dados.localidade, dados.estado, dados.uf, dados.cep
].filter(Boolean).join(", ");
mostrarCampo("Endereço completo disponível", endereco);

const busca = encodeURIComponent(`${endereco}, Brasil`);
linkMaps.href =
  `https://www.google.com/maps/search/?api=1&query=${busca}`;
linkMaps.hidden = false;
} catch (erro) {
  resultado.textContent = erro.name === "AbortError"
    ? "A consulta demorou demais. Tente novamente."
    : "Não foi possível consultar. Verifique a conexão " +
      "e tente novamente.";
} finally {
  clearTimeout(limite);
}
}

consultarEndereco();
```

Entenda o código

`filter(Boolean)` remove campos vazios e `join` reúne as partes do endereço. `encodeURIComponent` prepara o texto para o parâmetro `query`. A URL abre uma pesquisa no Google Maps e não exige chave de API. O limite de 10 segundos evita manter a tela indefinidamente em carregamento.

Orientações de teste

Execute o projeto em um servidor local, como o Live Server(se tiver, caso não possua pode testar apertando F5). Para cada teste, registre entrada, resultado esperado, resultado obtido e evidência em docs/testes.md. Use o modo offline do navegador para simular falha de rede.

Teste	Resultado esperado
01001-000 e São Paulo	Exibir endereço e liberar o mapa.
01001000 e sao paulo	Aceitar diferenças de acento, caixa e espaços.
12345 ou 01001A000	Rejeitar formato sem consultar a API.
CEP ou cidade em branco	Solicitar o preenchimento.
99999999 e São Paulo	Informar CEP não encontrado.
01001000 e Curitiba	Informar divergência e manter mapa oculto.
resultado.html sem parâmetros	Informar dados inválidos.
Rede offline ou consulta lenta	Apresentar falha ou tempo excedido.

7 Verificação final e entrega

Confira também um retorno com complemento vazio: deve aparecer “Não informado”. Teste o link do mapa, a volta para nova consulta, o uso pelo teclado e a tela na largura de 360 px. O mapa deve receber o endereço retornado, sem número de residência inventado.

Entrega via Git

Com o repositório inicializado, o remoto origin configurado e a branch main ativa, execute no terminal da pasta do projeto:

```
Entrega via Git  
git add .  
git commit -m "Finaliza consulta de endereço por CEP"  
git push origin main
```

Abra o repositório remoto e confirme que o último commit e os arquivos estão na branch main. Se o envio falhar, corrija a configuração ou a autenticação e repita o push antes de entregar.

Entregue o link do repositório com as duas páginas, CSS e JavaScript. Inclua no README as instruções para executar e o link ou a evidência do Trello. Mantenha em docs/ o diagrama de casos de uso, os dois esboços de UX e o registro dos testes.

Durante a apresentação individual, explique a validação do CEP, a passagem de parâmetros entre páginas, a requisição ao ViaCEP, a conferência de cidade e a construção do link do mapa.

Lista de conferência do estudante

Requisitos classificados e condições de aceite verificáveis.

Diagrama coerente com o sistema e esboços das duas páginas.

Fluxo funcional com mensagens de erro e campos ausentes tratados.

Testes registrados e Google Maps conferido.

Commit enviado para main e documentação acessível.

Ficha de avaliação individual

USO EXCLUSIVO DO DOCENTE • VALOR TOTAL 10 PONTOS

Estudante: _____
 Turma: _____ Data: _____
 Docente: _____
 Repositório: _____

Capacidades avaliadas

C1 — Interpretar requisitos levantados para desenvolvimento de sistemas.

C2 — Aplicar linguagem de programação para modelagem dos requisitos do sistema.

Cap.	Critério e evidência esperada	Máximo	Nota
C1	Requisitos claros, classificados e verificáveis no Trello.	1,5	
C1	Casos de uso com atores, fronteira e associações coerentes.	1,0	
C1	UX das duas páginas e estados previstos nos esboços.	1,0	
C2	Captura e validação dos dados; navegação entre páginas e validação da URL.	1,5	
C2	Consumo do ViaCEP, conferência de cidade e tratamento de falhas.	2,0	
C2	Endereço disponível completo, campos ausentes e link do Maps corretos.	1,0	
C2	Testes registrados e explicação individual do funcionamento.	1,0	
C2	Arquivos e documentação entregues com commit e push na main.	1,0	
	TOTAL	10,0	

Pontuação por critério: atribua 100% do valor quando atendido, 50% quando parcialmente atendido e 0 quando não atendido ou sem evidência. Registre a justificativa dos descontos. Avalie a contribuição e a compreensão de cada estudante, mesmo em trabalho em equipe.

Evidências e devolutiva individual:

Nota final: _____ / 10 Assinatura do docente: _____